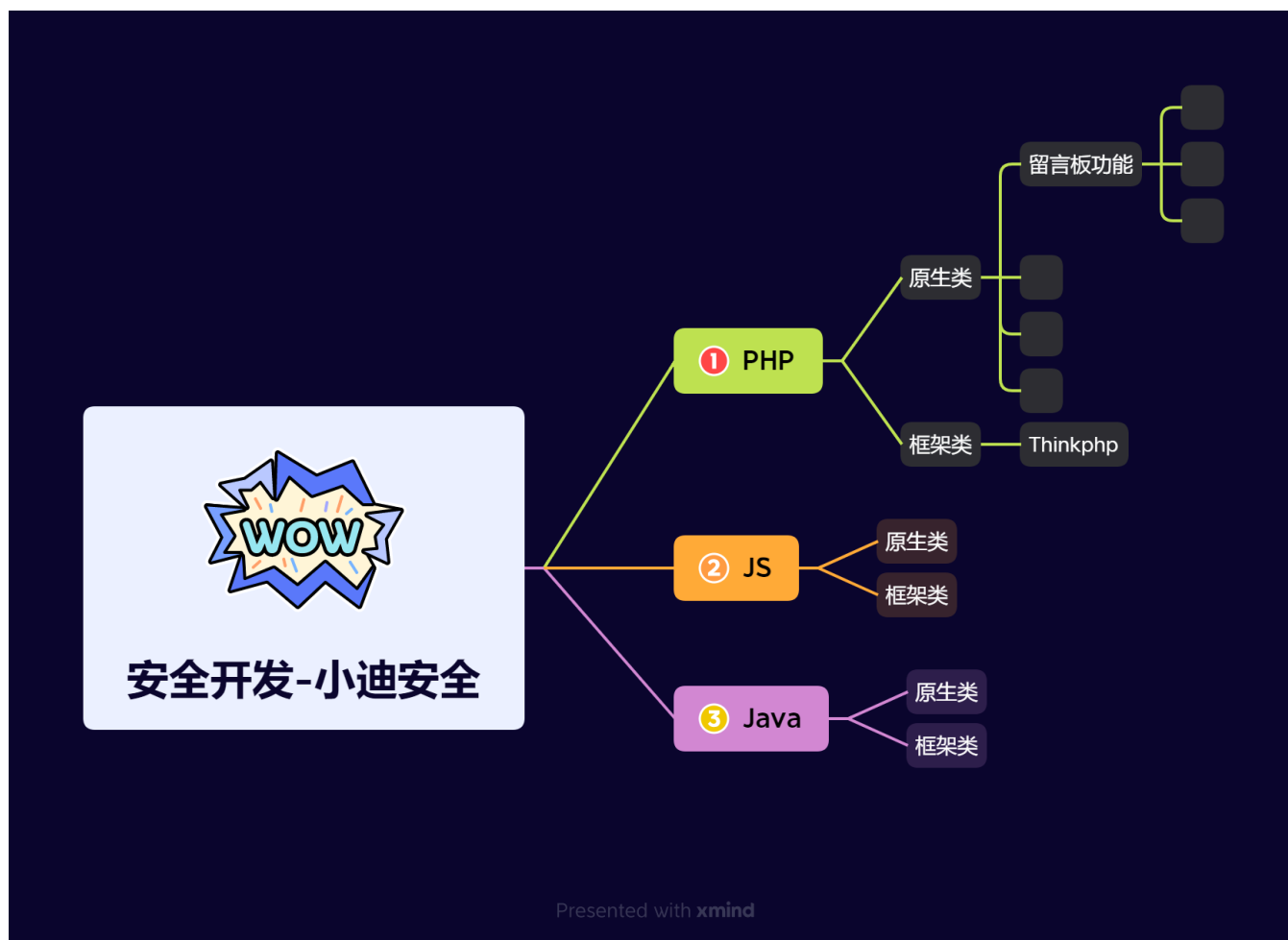


Day30 安全开发-JS应用&NodeJS指南&原型链污染&Express框架&功能实现&审计



1.知识点

- 1、PHP留言板前后端功能实现
- 2、数据库创建&架构&增删改查
- 3、内置超全局变量&HTML&JS混编
- 4、第三方应用插件&传参&对象调用

-
- 1、PHP后台身份验证模块实现
 - 2、Cookie&Session技术&差异
 - 3、Token数据包唯一性应用场景
 - 项目1：用cookie做后台身份验证
 - 项目2：用session做后台身份验证
 - 项目3：用token做用户登录判断
-

- 1、PHP文件管理-显示&上传功能实现
 - 2、文件上传-\$_FILES&过滤机制实现
 - 3、文件显示-目录遍历&过滤机制实现
-

- 1、PHP文件管理-下载&删除功能实现
 - 2、PHP文件管理-编辑&包含功能实现
-

- 1、PHP新闻显示-数据库操作读取显示
 - 2、PHP模版引用-自写模版&Smarty渲染
 - 3、PHP模版安全-RCE代码执行&三方漏洞
-

- 1、TP框架-开发-路由访问&数据库&文件上传&MVC模型
 - 2、TP框架-安全-不合规写法&内置过滤绕过&版本安全漏洞
-

- 1、JS应用-原生态开发&第三库开发
 - 2、JS功能-文件上传&登录验证&商品购买
-

- 1、JS技术-DOM树操作及安全隐患
 - 2、JS技术-加密编码及数据安全调试
-

- 1、NodeJS-开发环境&功能实现
 - 2、NodeJS-安全漏洞&案例分析
 - 3、NodeJS-开发指南&特有漏洞
-

2.演示案例

2.1 环境搭建-NodeJS-解析安装&库安装



```
1 0、文档参考：
2 https://www.w3cschool.cn/nodejs/
3 1、Nodejs安装
4 https://nodejs.org/en
5
6 2、三方库安装
7 express
8 Express是一个简洁而灵活的node.js web应用框架
9
```

```
10 body-parser
11 node.js中间件，用于处理 JSON, Raw, Text和URL编码的数据。
12
13 cookie-parser
14 这就是一个解析Cookie的工具。通过req.cookies可以取到传过来的cookie，并把它们转成对象。
15
16 multer
17 node.js中间件，用于处理 enctype="multipart/form-data"（设置表单的MIME编码）的表单数据。
18
19 mysql
20 Node.js来连接MySQL专用库，并对数据库进行操作。
21
22 安装命令：
23 npm i express
24 npm i body-parser
25 npm i cookie-parser
26 npm i multer
27 npm i mysql
```

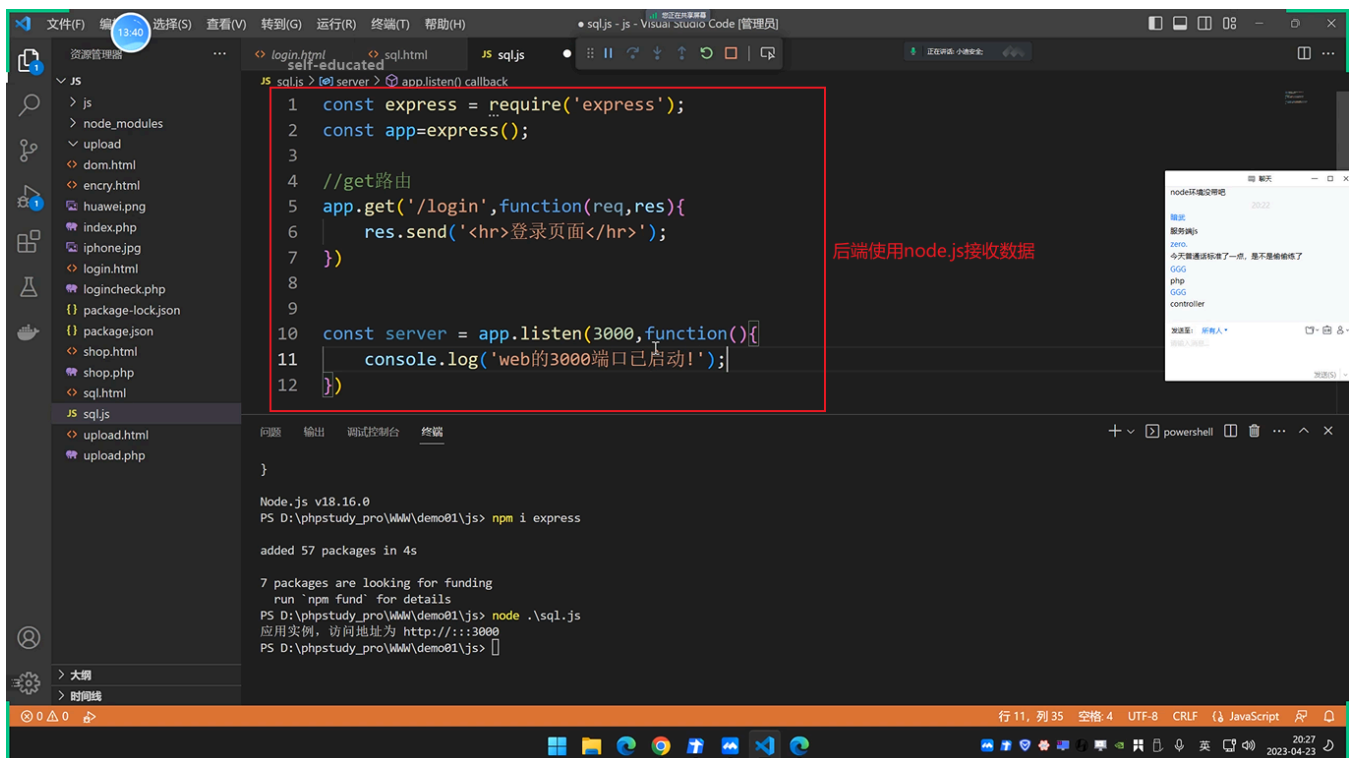
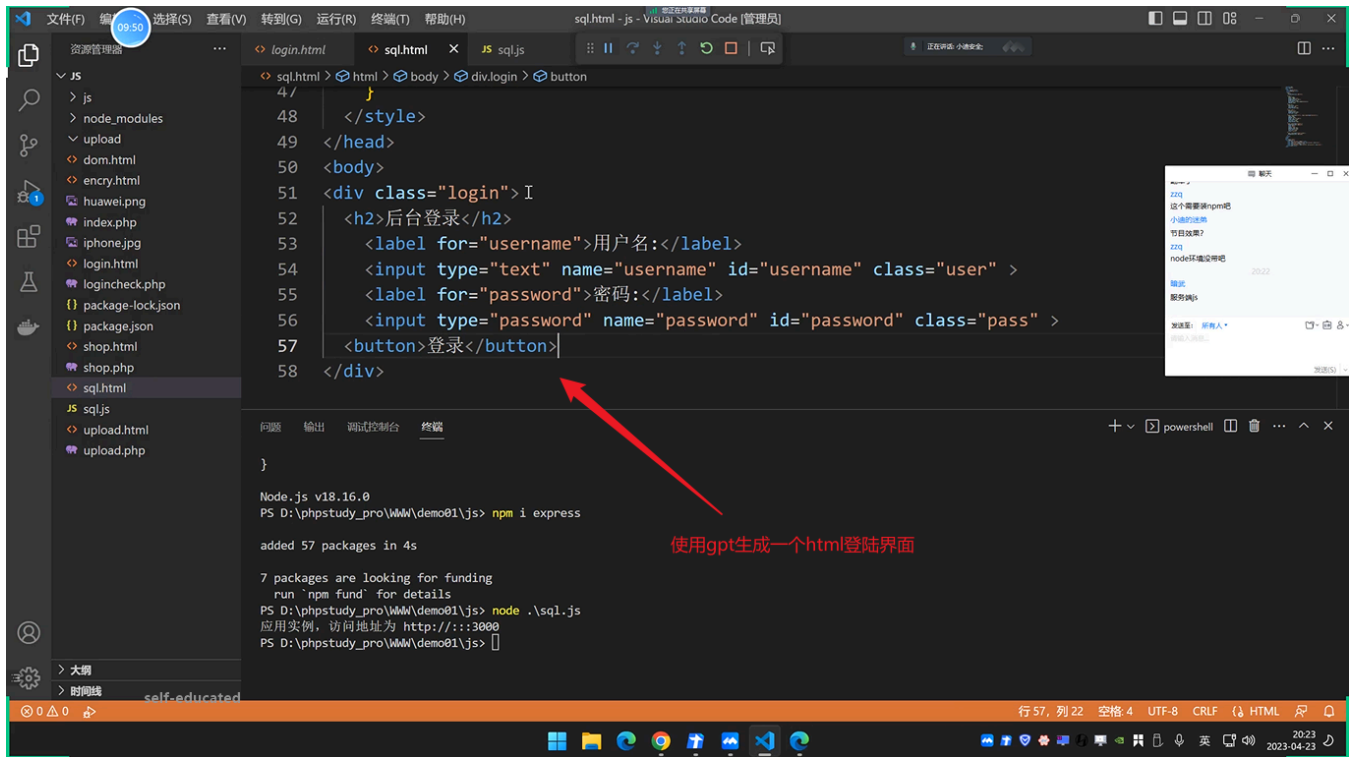
2.2 功能实现-NodeJS-数据库&文件&执行

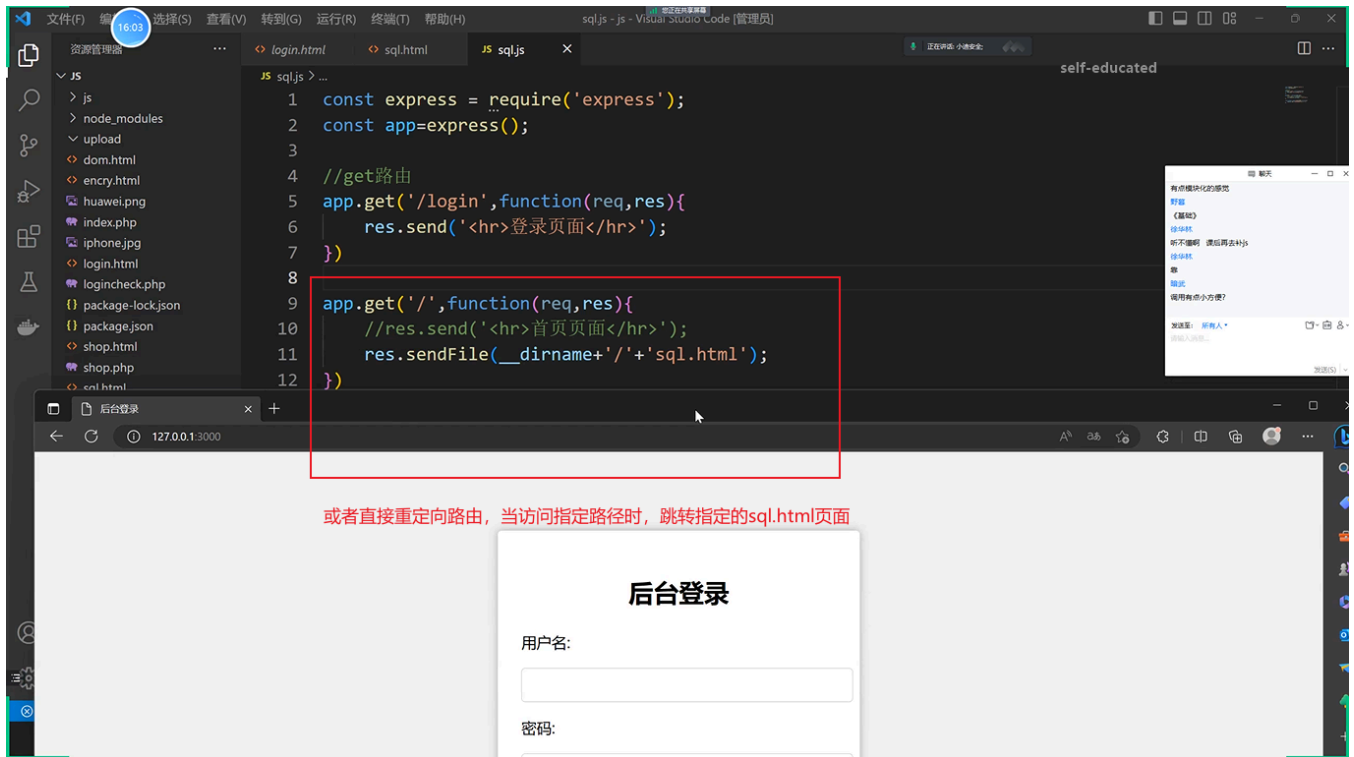


```
1 -登录操作
2 1、Express开发
3 2、实现用户登录
4 3、加入数据库操作
5
6 -文件操作
7 1、Express开发
8 2、实现目录读取
9 3、加入传参接受
10
11 -命令执行（RCE）
12 1、eval
13 2、exec & spawnSync
```

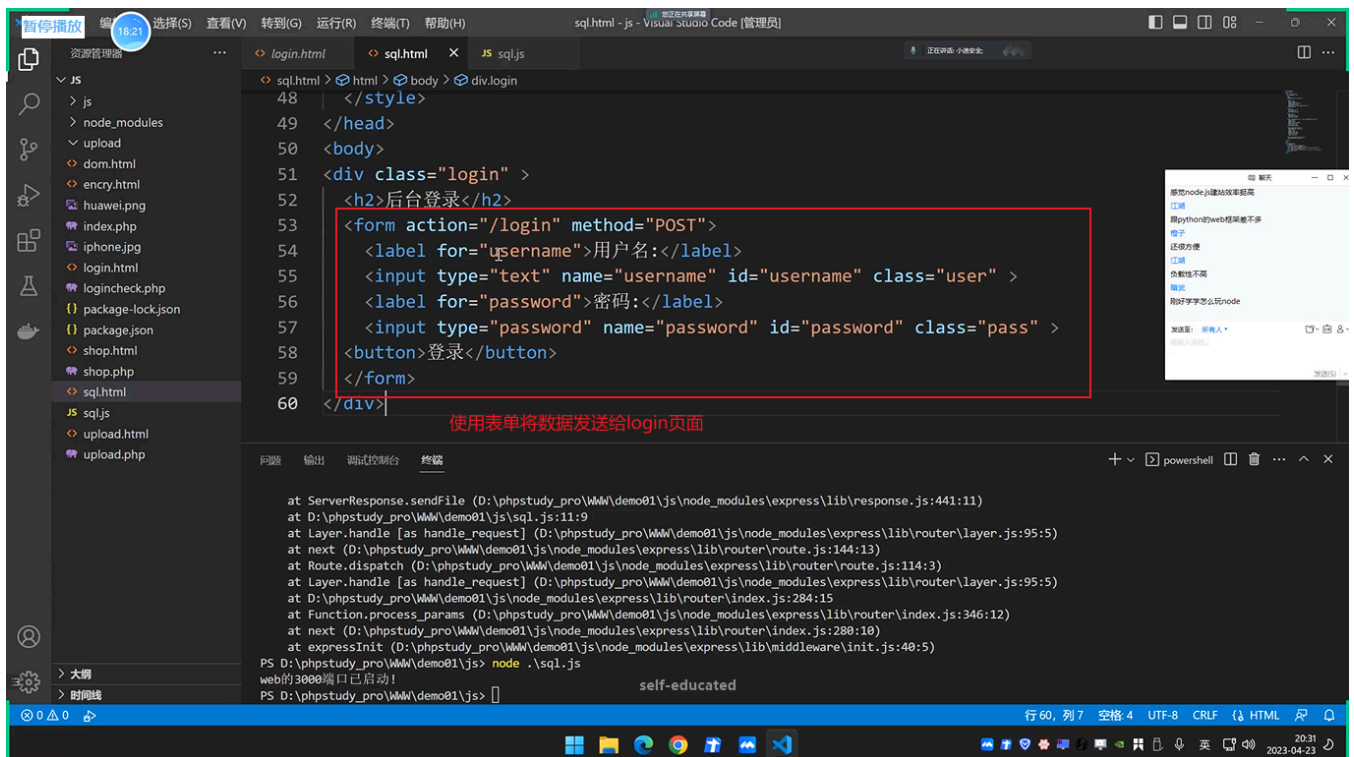
2.2.1 登录操作

(1) 实现登录操作：





(2) 接收sql.html传递过来的参数值:



文件(F) 编辑(E) 选择(S) 查看(V) 转到(G) 运行(R) 终端(T) 帮助(H) • sql.js - Visual Studio Code [管理员]

资源管理器 19:23

JS sql.js

```
4 //get路由
5 app.get('/login',function(req,res){
6
7 })
8
9 app.get('/',function(req,res){
10 //res.send('<hr>首页面</hr>');
11 res.sendFile(__dirname+'/'+'.sql.html');
12 })
13
14 const server = app.listen(3000,function(){
15 console.log('web的3000端口已启动!');
16 })
```

接收数据

显示页面

启动服务

问题 输出 调试控制台 终端

```
at ServerResponse.sendFile (D:\phpstudy_pro\WWW\demo01\js\node_modules\express\lib\Response.js:441:11)
at D:\phpstudy_pro\WWW\demo01\js\sql.js:11:9
at Layer.handle [as handle_request] (D:\phpstudy_pro\WWW\demo01\js\node_modules\express\lib\router\layer.js:95:5)
at next (D:\phpstudy_pro\WWW\demo01\js\node_modules\express\lib\router\route.js:144:13)
at Route.dispatch (D:\phpstudy_pro\WWW\demo01\js\node_modules\express\lib\router\route.js:114:3)
at Layer.handle [as handle_request] (D:\phpstudy_pro\WWW\demo01\js\node_modules\express\lib\router\layer.js:95:5)
at D:\phpstudy_pro\WWW\demo01\js\node_modules\express\lib\router\index.js:284:15
at Function.process_params (D:\phpstudy_pro\WWW\demo01\js\node_modules\express\lib\router\index.js:346:12)
at next (D:\phpstudy_pro\WWW\demo01\js\node_modules\express\lib\router\index.js:280:10)
at expressInit (D:\phpstudy_pro\WWW\demo01\js\node_modules\express\lib\middleware\init.js:40:5)
PS D:\phpstudy_pro\WWW\demo01\js> node .\sql.js
web的3000端口已启动!
PS D:\phpstudy_pro\WWW\demo01\js>
```

self-educated

行 6, 列 5 空格 4 UTF-8 CRLF JavaScript

20:32 2023-04-23

暂停播放 19:23 选择(S) 查看(V) 转到(G) 运行(R) 终端(T) 帮助(H) sql.js - Visual Studio Code [管理员]

资源管理器 self-educated

JS sql.js

```
4 //get路由
5 app.get('/login',function(req,res){
6 const u = req.query.username;
7 const p = req.query.password;
8 if(u=='admin' && p == '123456'){
9 console.log(u);
10 console.log(p);
11 res.send('欢迎进入后台管理页面');
12 }else{
13 res.send('登录用户或密码错误!');
14 }
15 })
16
17 app.get('/',function(req,res){
```

接收数据并给用户提示

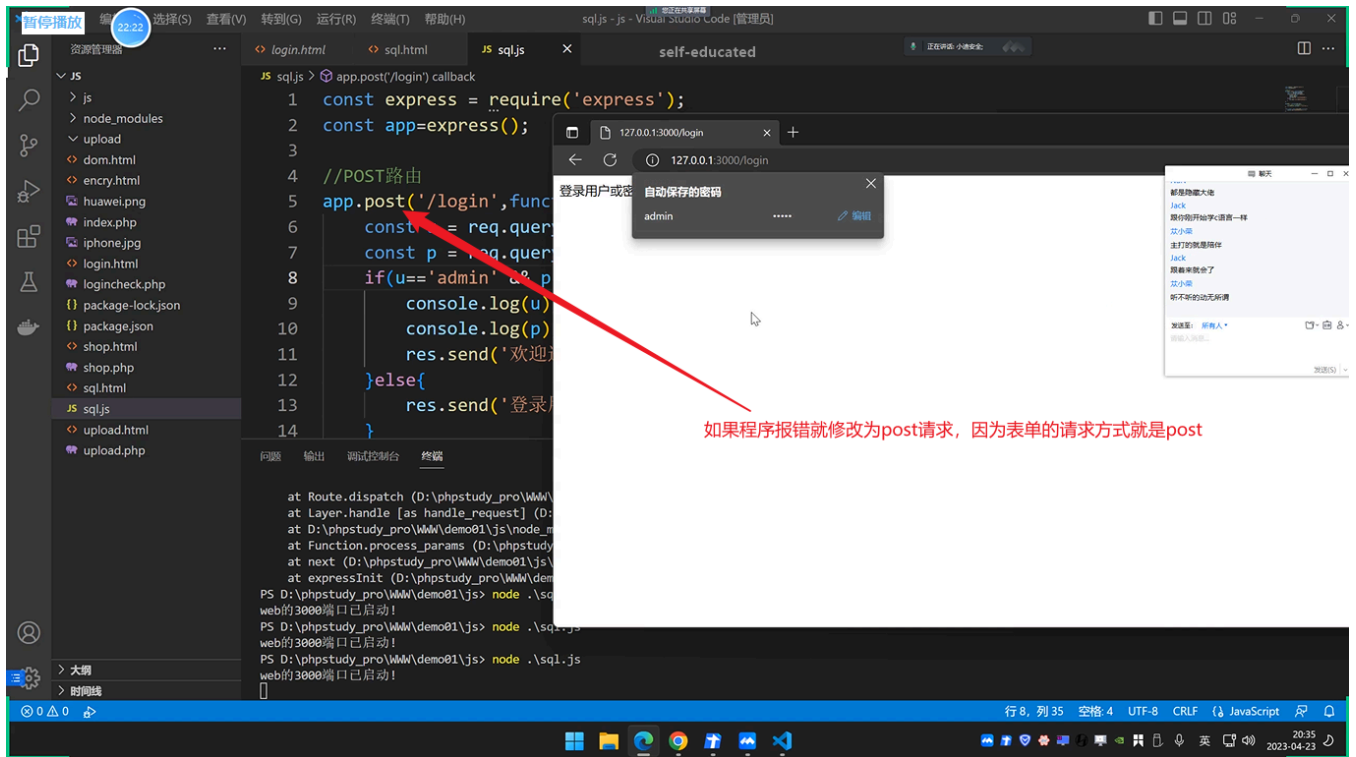
问题 输出 调试控制台 终端

```
at ServerResponse.sendFile (D:\phpstudy_pro\WWW\demo01\js\node_modules\express\lib\Response.js:441:11)
at D:\phpstudy_pro\WWW\demo01\js\sql.js:11:9
at Layer.handle [as handle_request] (D:\phpstudy_pro\WWW\demo01\js\node_modules\express\lib\router\layer.js:95:5)
at next (D:\phpstudy_pro\WWW\demo01\js\node_modules\express\lib\router\route.js:144:13)
at Route.dispatch (D:\phpstudy_pro\WWW\demo01\js\node_modules\express\lib\router\route.js:114:3)
at Layer.handle [as handle_request] (D:\phpstudy_pro\WWW\demo01\js\node_modules\express\lib\router\layer.js:95:5)
at D:\phpstudy_pro\WWW\demo01\js\node_modules\express\lib\router\index.js:284:15
at Function.process_params (D:\phpstudy_pro\WWW\demo01\js\node_modules\express\lib\router\index.js:346:12)
at next (D:\phpstudy_pro\WWW\demo01\js\node_modules\express\lib\router\index.js:280:10)
at expressInit (D:\phpstudy_pro\WWW\demo01\js\node_modules\express\lib\middleware\init.js:40:5)
PS D:\phpstudy_pro\WWW\demo01\js> node .\sql.js
web的3000端口已启动!
PS D:\phpstudy_pro\WWW\demo01\js>
```

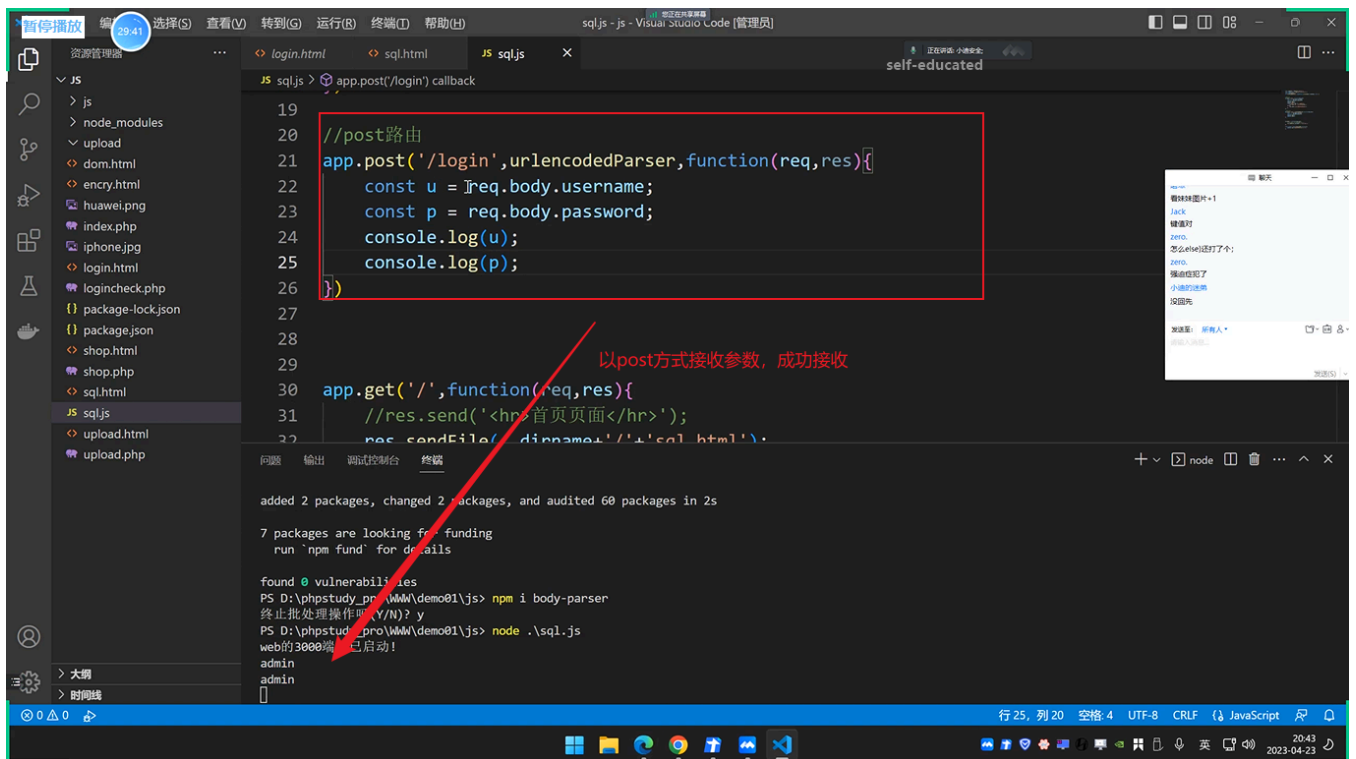
self-educated

行 13, 列 32 空格 4 UTF-8 CRLF JavaScript

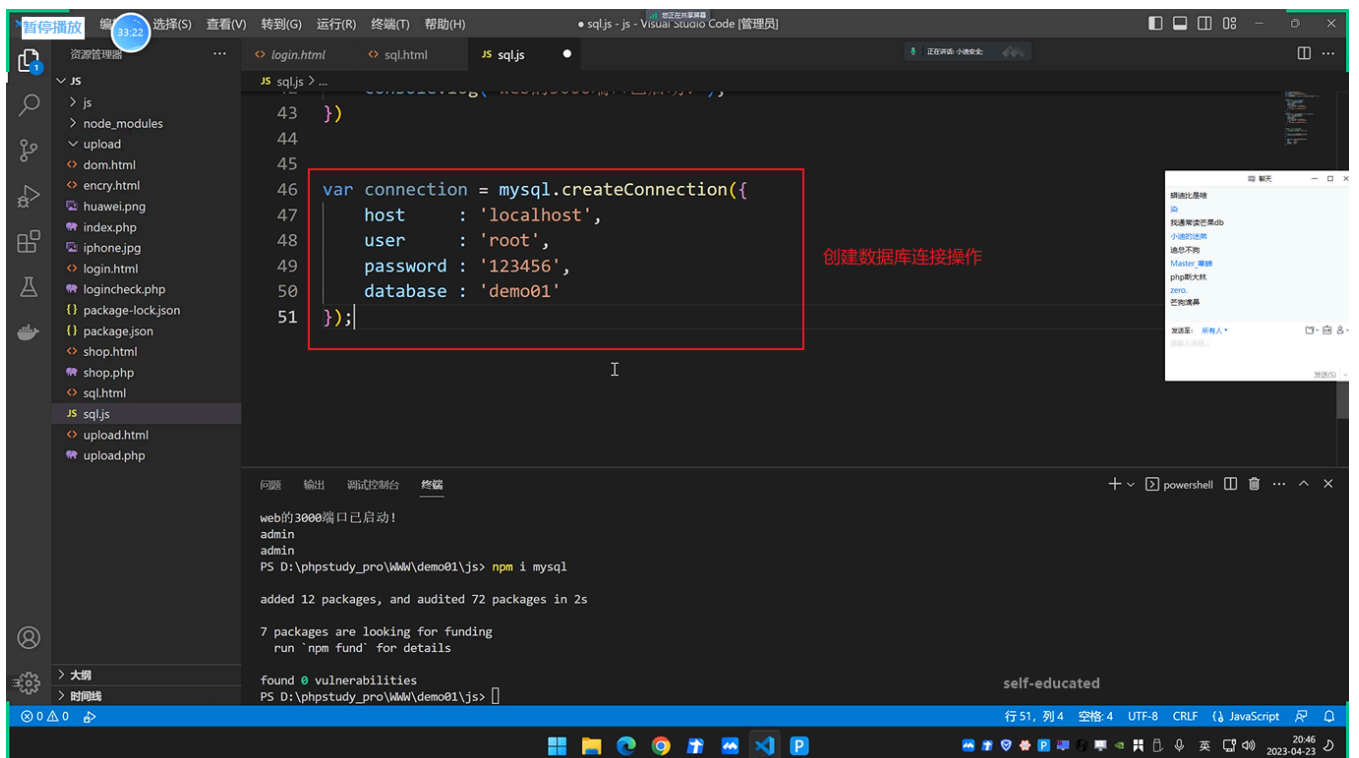
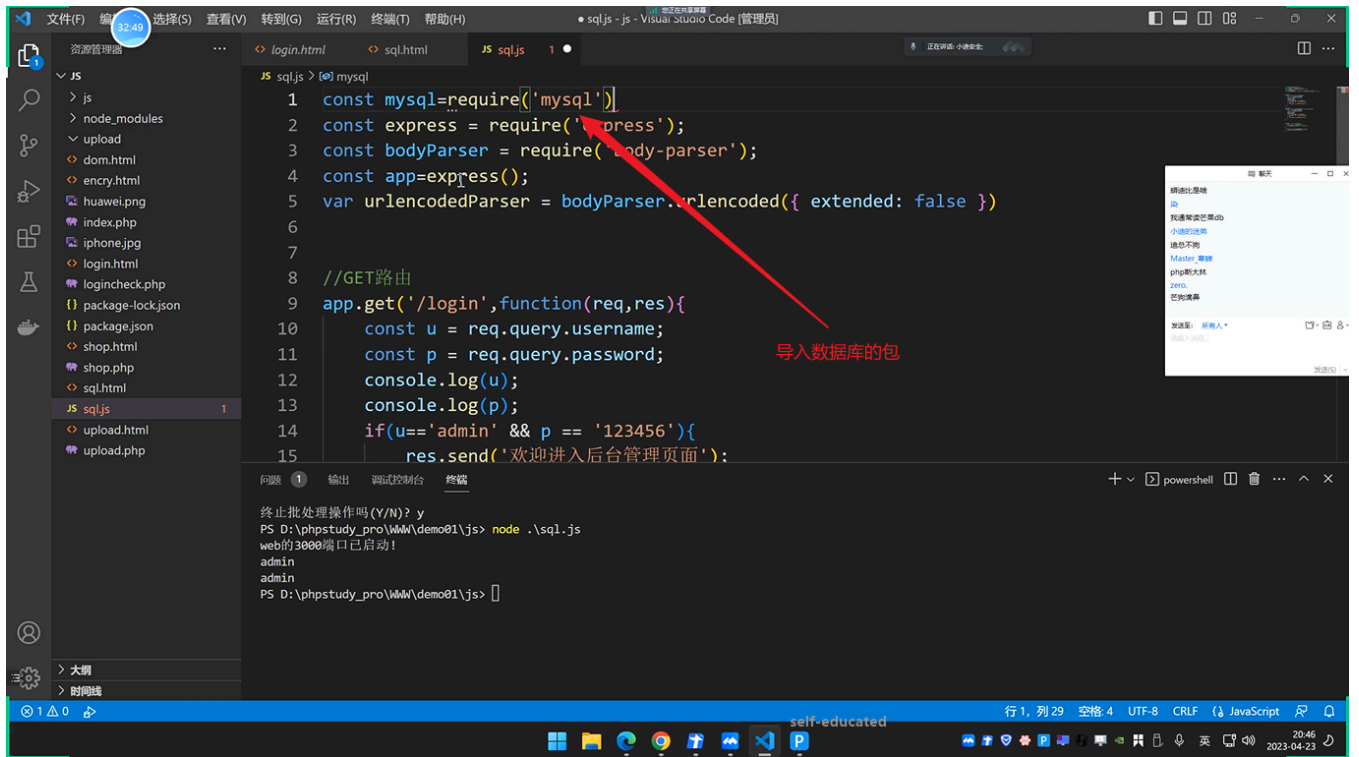
20:34 2023-04-23

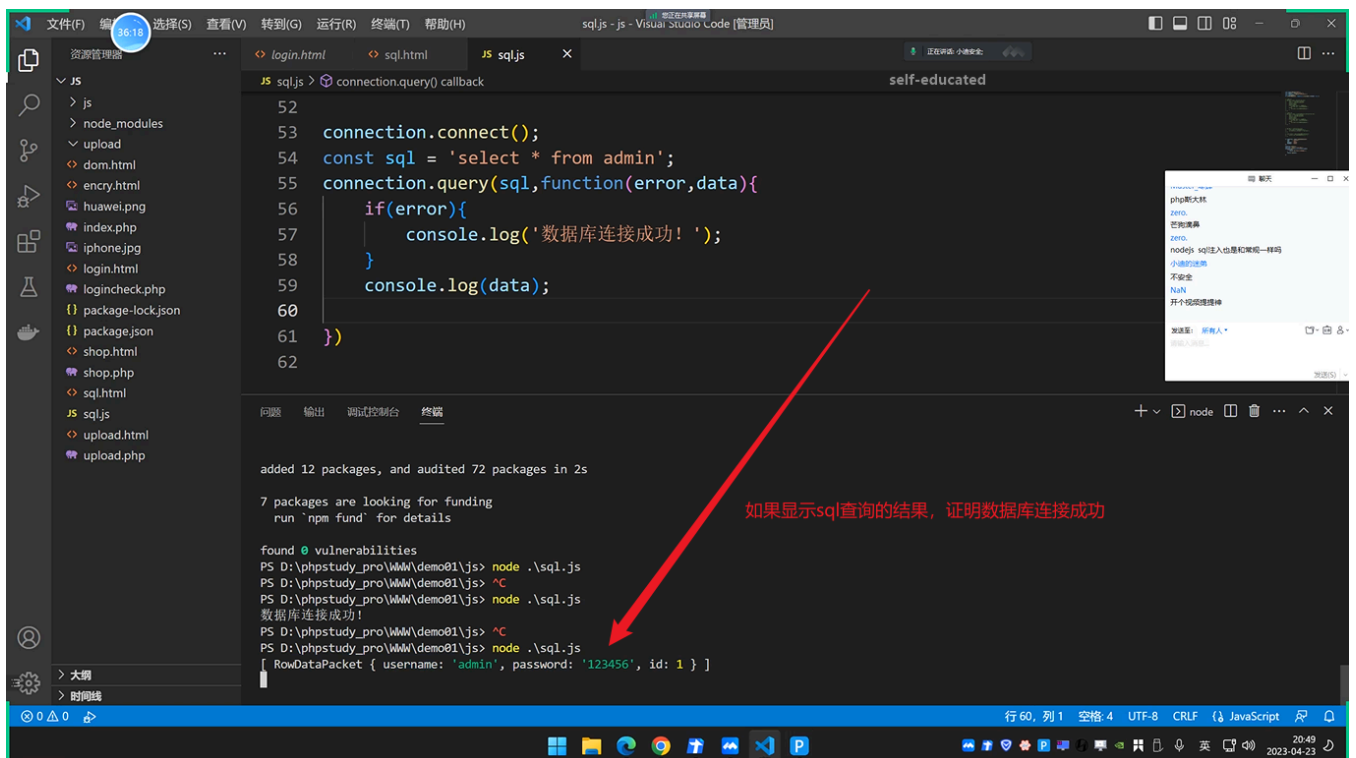
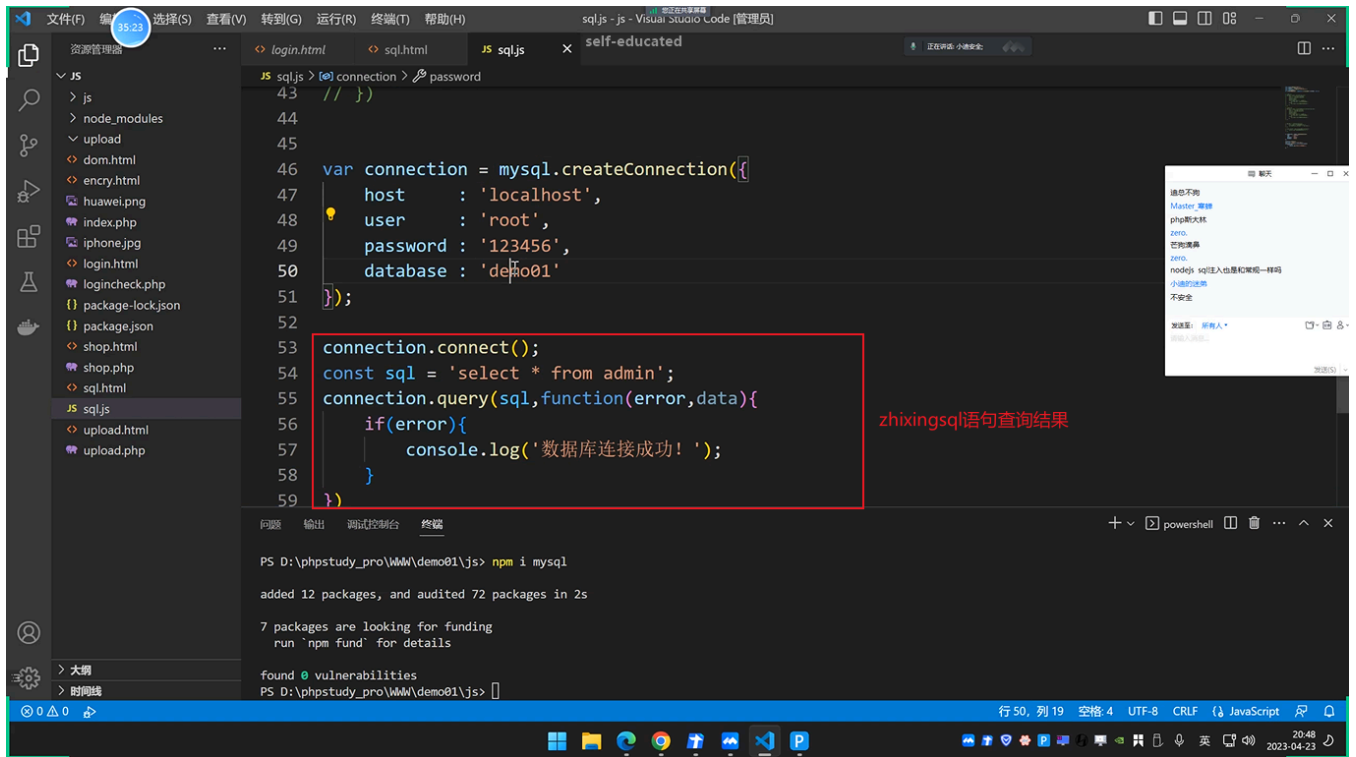


(3) 如果使用get请求上述程序可以直接运行，如果使用post请求，则还需要引用库：



(4) 联动数据库进行账号密码校验：





41:45

文件(F) 编辑(E) 选择(S) 查看(V) 转到(G) 运行(R) 终端(T) 帮助(H)

login.html sql.html JS sql.js

JS sql.js > app.post('/login') callback > connection self-educated

20

21 //post路由

22 app.post('/login',urlencodedParser,function(req,res){

23 const u = req.body.username;

24 const p = req.body.password;

25 console.log(u);

26 console.log(p);

27 var connection = mysql.createConnection({

28 host : 'localhost',

29 user : 'root',

30 password : '123456',

31 database : 'demo01'

32 });

33

34 connection.connect();

35 const sql = 'select * from admin where username='+u+' and password='+p;

36 console.log(sql);

37 connection.query(sql,function(error,data){

38 if(error){

问题 输出 调试控制台 终端

code: 'PROTOCOL_CONNECTION_LOST'

Node.js v18.16.0

PS D:\phpstudy_pro\WWW\demo01\js>

行 27, 列 25 空格 4 UTF-8 CRLF JavaScript

20:55 2023-04-23

修改post提交的代码
在接收参数以后建立数据库连接
判断接受的值是否与数据库中的值相等

42:56

文件(F) 编辑(E) 选择(S) 查看(V) 转到(G) 运行(R) 终端(T) 帮助(H)

login.html sql.html JS sql.js

JS sql.js > app.post('/login') callback > sql self-educated

25 console.log(u);

26 console.log(p);

27 var connection = mysql.createConnection({

28 host : 'localhost',

29 user : 'root',

30 password : '123456',

31 database : 'demo01'

32 });

33

34 connection.connect();

35 const sql = 'select * from admin where username="'+u+'" and password="'+p+'';

36 console.log(sql);

37 connection.query(sql,function(error,data){

38 if(error){

问题 输出 调试控制台 终端

at Parser._parsePacket (D:\phpstudy_pro\WWW\demo01\js\node_modules\mysql\lib\protocol\Parser.js:433:10)

at Parser.write (D:\phpstudy_pro\WWW\demo01\js\node_modules\mysql\lib\protocol\Parser.js:43:10)

at Protocol.write (D:\phpstudy_pro\WWW\demo01\js\node_modules\mysql\lib\protocol\Protocol.js:38:16)

at Socket.<anonymous> (D:\phpstudy_pro\WWW\demo01\js\node_modules\mysql\lib\Connection.js:88:28)

Node.js v18.16.0

PS D:\phpstudy_pro\WWW\demo01\js> node .\sql.js

web的3000端口已启动!

admin

123456

Select * from admin where username="admin" and password="123456"

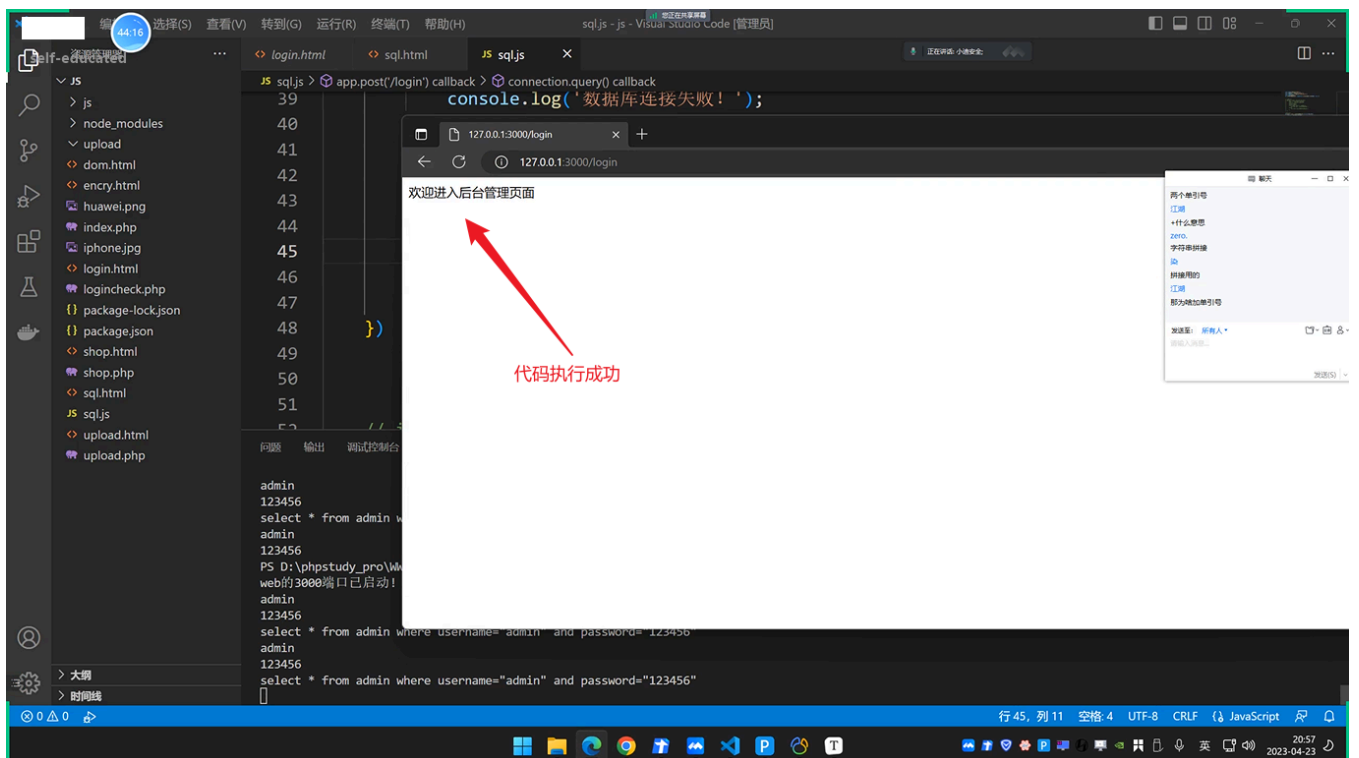
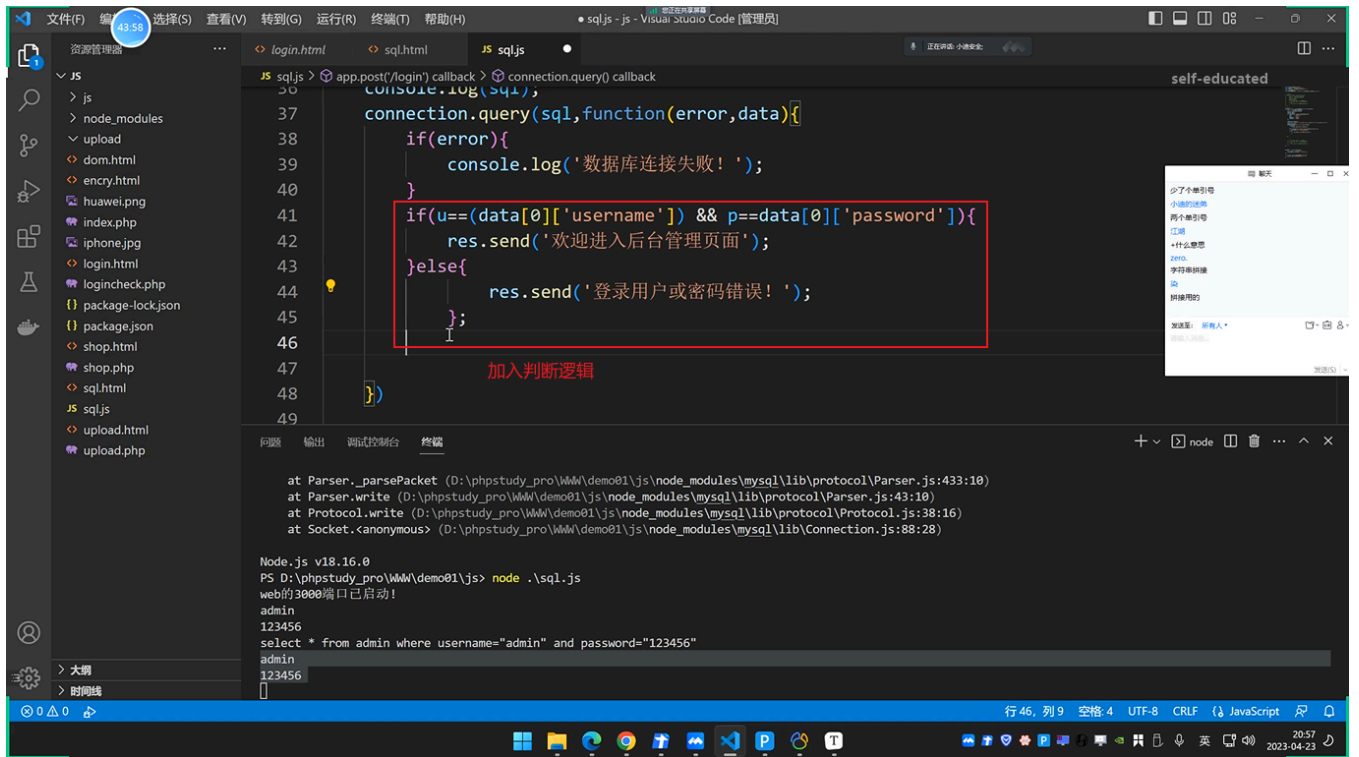
admin

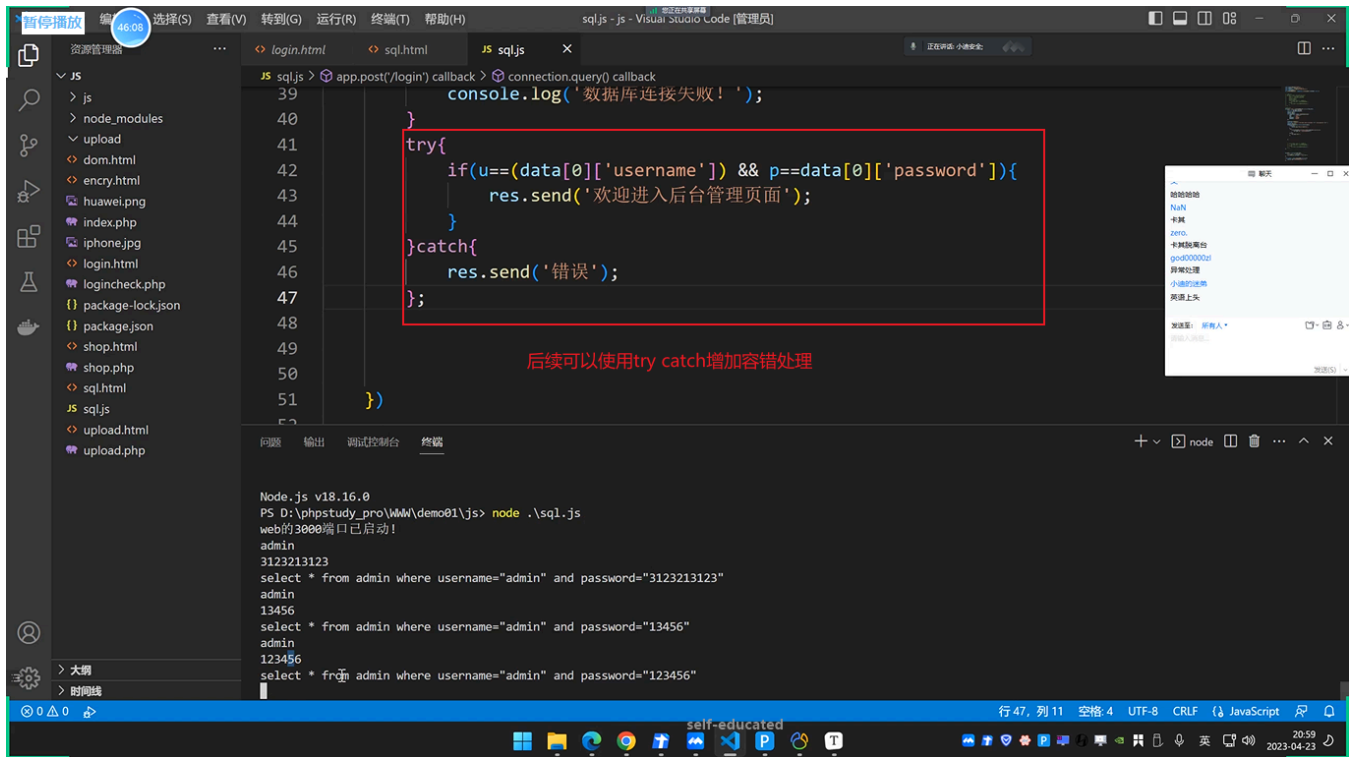
123456

成功接收到参数

行 35, 列 62 空格 4 UTF-8 CRLF JavaScript

20:56 2023-04-23

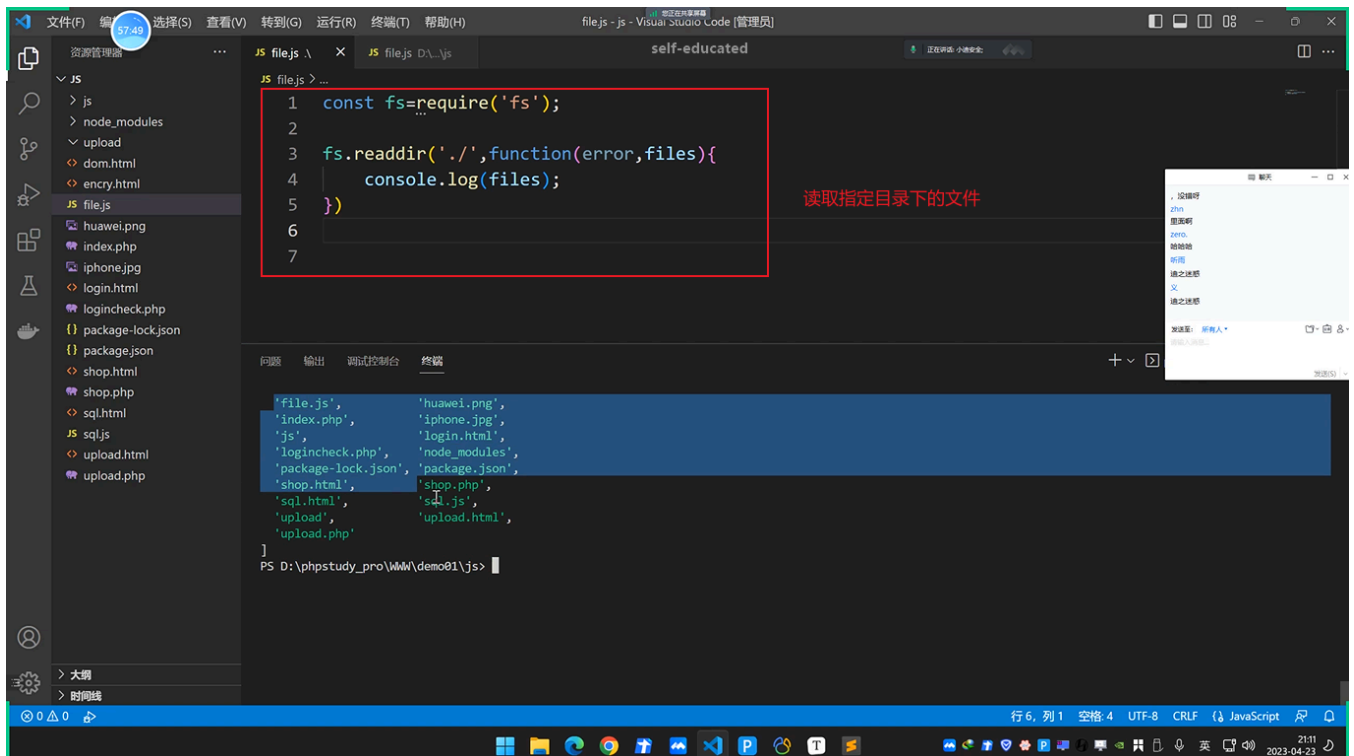


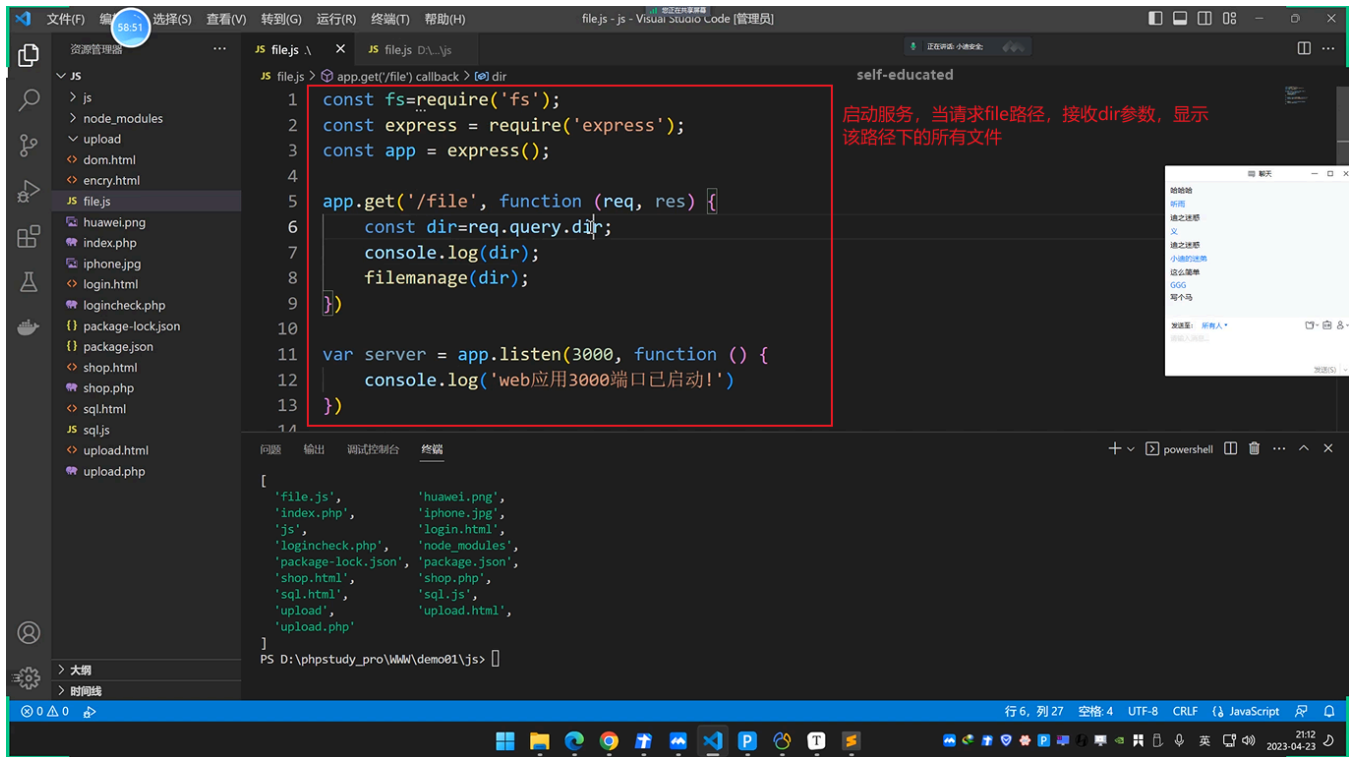


- 1 在上述代码中存在sql注入。

2.2.2 文件操作

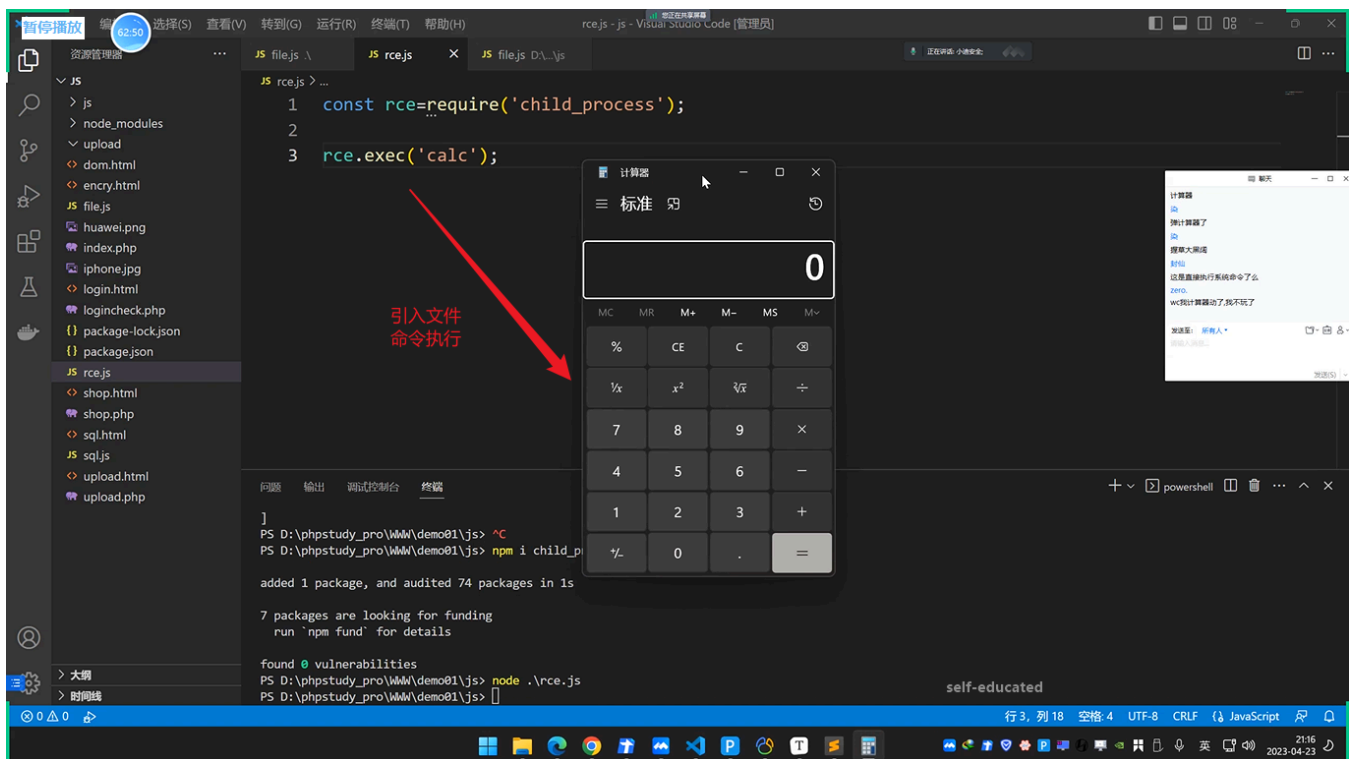
(1) 编写代码实现文件遍历：

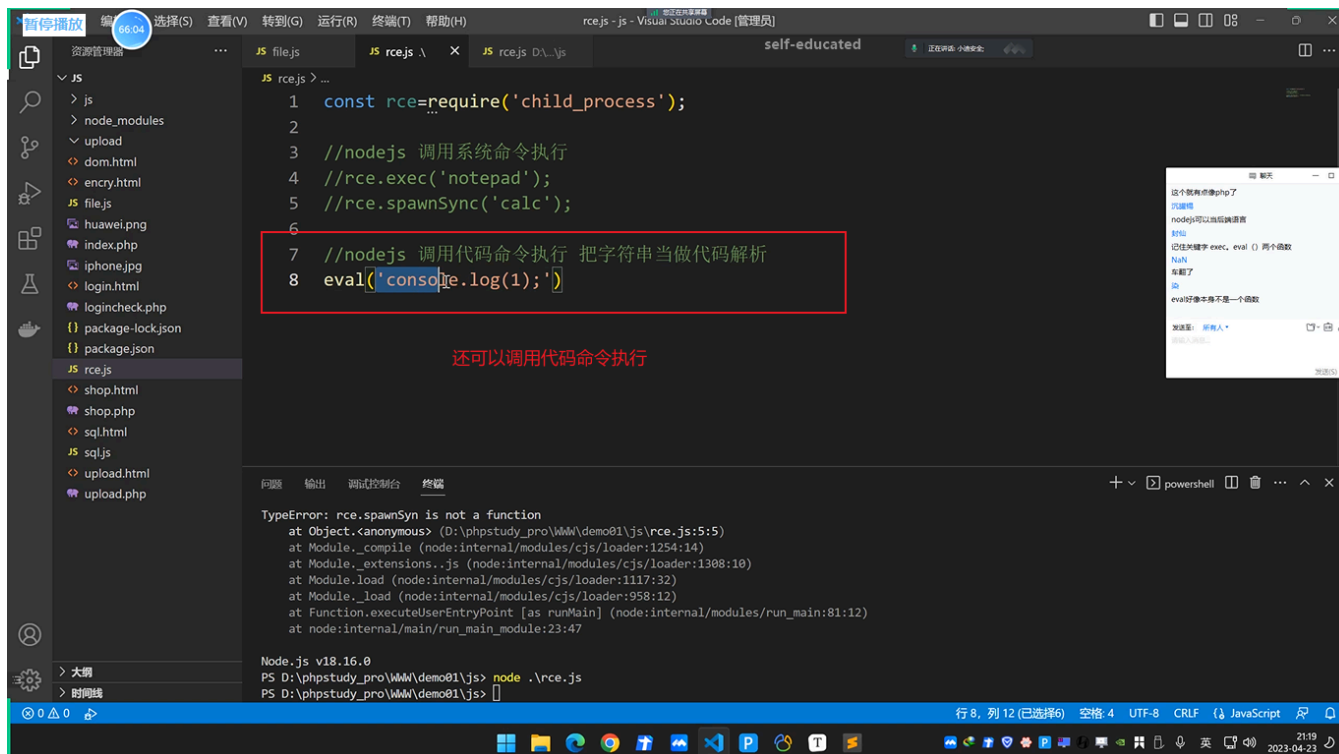




2.2.3 命令执行 (RCE)

(1) 实现命令执行操作:





2.3 安全问题-NodeJS-注入&RCE&原型链

- 1、SQL注入&文件操作
- 2、RCE执行&原型链污染
- 2、NodeJS黑盒无代码分析
- 实战测试NodeJS安全:
- 判断: 参考前期的信息收集
- 黑盒: 通过对各种功能和参数进行payload测试
- 白盒: 通过对代码中写法安全进行审计分析
- 原型链污染
- 如果攻击者控制并修改了一个对象的原型, (__proto__)
- 那么将可以影响所有和这个对象来自同一个类、父祖类的对象。

(1) 原型链污染:

```
1 // foo是一个简单的JavaScript对象
2 let foo = {bar: 1}
3 // 原型链污染
4 // foo.bar 此时为1
5 console.log(foo.bar)
6
7 // 修改foo的原型 (即Object)
8 // foo.__proto__.bar = 2
9
10 // // 由于查找顺序的原因, foo.bar仍然是1
11 // console.log(foo.bar)
12
13 // // 此时再用Object创建一个空的zoo对象
14 // let zoo = {}
15
```

给foo变量赋值, 此时为1

```
at Object.<anonymous> (D:\phpstudy_pro\WWW\demo01\js\y-rce.js:17:13)
at Module._compile (node:internal/modules/cjs/loader:1254:14)
at Module._extensions..js (node:internal/modules/cjs/loader:1308:10)
at Module.load (node:internal/modules/cjs/loader:1117:32)
at Module._load (node:internal/modules/cjs/loader:958:12)
at Function.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:81:12)
at node:internal/main/run_main_module:23:47

Node.js v18.16.0
PS D:\phpstudy_pro\WWW\demo01\js> node .\y-rce.js
1
PS D:\phpstudy_pro\WWW\demo01\js>
```

```
1 // foo是一个简单的JavaScript对象
2 let foo = {bar: 1}
3 // 原型链污染
4 // foo.bar 此时为1
5 console.log(foo.bar)
6
7 // 修改foo的原型 (即Object)
8 foo.__proto__.bar = 2
9
10 // // 由于查找顺序的原因, foo.bar仍然是1
11 console.log(foo.bar)
12
13 // // 此时再用Object创建一个空的zoo对象
14 // let zoo = {}
15
```

此时修改参数的值发现还是1

```
at Module.load (node:internal/modules/cjs/loader:1117:32)
at Module._load (node:internal/modules/cjs/loader:958:12)
at Function.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:81:12)
at node:internal/main/run_main_module:23:47

Node.js v18.16.0
PS D:\phpstudy_pro\WWW\demo01\js> node .\y-rce.js
1
PS D:\phpstudy_pro\WWW\demo01\js> node .\y-rce.js
1
PS D:\phpstudy_pro\WWW\demo01\js>
```


2.4 案例分析-NodeJS-CTF题目&源码审计



- 1 1、CTFSHOW几个题目
- 2 <https://ctf.show/> web334-344
- 3 <https://f1veseven.github.io/2022/04/03/ctf-nodejs-zhi-yi-xie-xiao-zhi-shi/>
- 4 2、YApi管理平台漏洞
- 5 https://blog.csdn.net/weixin_42353842/article/details/127960229
- 6
- 7 #开发指南-NodeJS-安全SecGuide项目
- 8 <https://github.com/Tencent/secguide>